Методологии разработки ПО

Методология разработки

Методология разработки — это набор принципов, ролей и практик, регулирующих процесс создания программы. Они предполагают тщательное планирование жизненного цикла разработки. Но есть важная особенность — они не прописывают конкретные технические моменты реализации.

Методологии учитывают управление проектом, сотрудничество и коммуникацию, а не технические детали, такие как выбор языка программирования или базы данных. Команды используют их, чтобы работать последовательно, выпускать продукты точно в срок и избегать недопонимания между членами команды. Если использовать методологии, то можно избежать таких ситуаций.

Методологии служат структурной основой для инициирования, выполнения и запуска IT проектов

Методология Waterfall



Методология Waterfall

Водопадная модель разработки — это линейный подход к разработке программного обеспечения. Она характеризуется строгим, структурированным процессом. Основной принцип — каждый этап проекта должен быть завершен до начала следующего. Как правило, модель состоит из следующих этапов разработки: системного анализа, анализа требований, проектирования, реализации, тестирования, внедрения и сопровождения.

Эта методология подходит для хорошо продуманных проектов, в которых требования строго зафиксированы и вряд ли сильно изменятся в процессе разработки. Она предлагает четкие рамки проекта, что облегчает управление ресурсами и распределение времени.

Методология Waterfall Плюсы

- Эта методология хорошо подходит для проектов со строгими и понятными требованиями.
- Может применяться для проектов любого масштаба — от небольших до крупных.
- Много внимания уделяется тщательному документированию каждого этапа разработки.
- Водопадная модель простая, четкая и понятная. Её легко внедрить в рабочий процесс благодаря простым фазам, которые идут строго по порядку.

Методология Waterfall Минусы

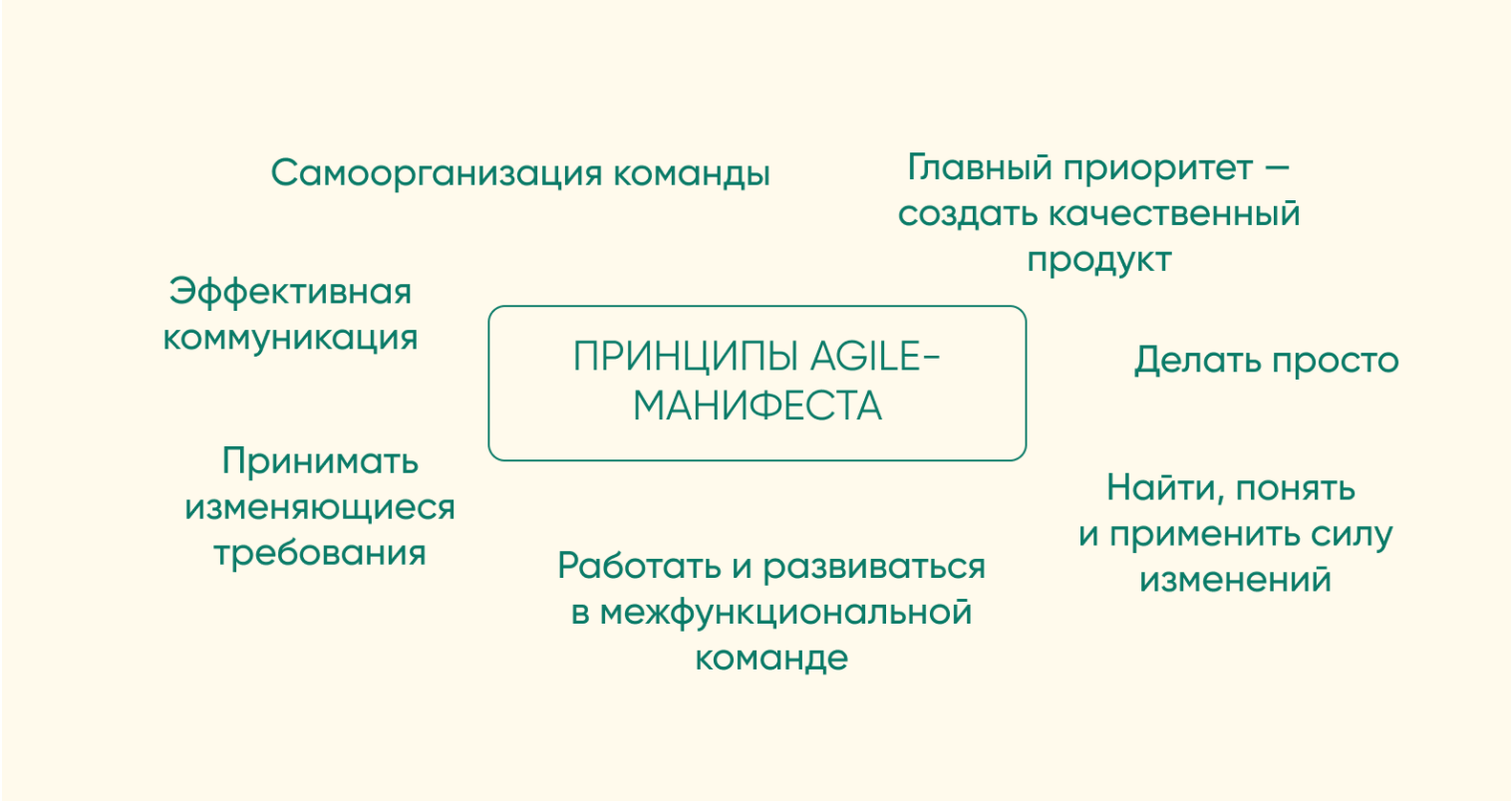
- Недостаточно гибкости. После завершения какого-либо этапа сложно вернуться назад и внести изменения, не нарушая последующие этапы.
- Может быть проблематично применять в проектах, где нужны итерации и частая обратная связь от владельца продукта.
- Предполагает ограниченную вовлеченность владельца продукта, поскольку он участвует в проекте в основном в начале и в конце его реализации.
- Фаза тестирования наступает в конце проекта, поэтому проблемы и ошибки могут быть выявлены не сразу — а устранять их будет дорого и сложно.

Методология Waterfall Кому подходит

✅ Водопадная модель лучше всего подходит для проектов с ясной конечной целью и четко определенными требованиями, которые вряд ли изменятся по ходу разработки. Она будет хорошо работать, если технологии, инструменты и процессы хорошо отлажены и не меняются. Методология может быть эффективна при реализации крупномасштабных проектов, требующих планирования и документации.

❌ Однако эта модель не очень хорошо подходит для проектов с меняющимися целями и сроками. Она недостаточно гибкая, поэтому что-то поменять после завершения одного из этапов может быть сложно. Водопадная модель не подойдет проектам, связанным с исследованиями, экспериментами и инновациями — они часто требуют правок и доработок.

Методология Agile



Методология Agile

Методология Agile — это популярный подход, в котором основное внимание уделяется гибкости, сотрудничеству и оптимизации процессов для реализации качественного проекта. Это итеративный подход, и приоритет в нем отдается обратной связи от владельца продукта и адаптации к изменяющимся требованиям. Цикл разработки ПО по Agile-методологии можно разбить на шесть этапов: планирование, проектирование, разработка, тестирование, развертывание и обслуживание.

Этот подход к разработке широко распространен как в IT-индустрии, так и в других областях — в управлении проектами, разработке продуктов и даже в проектах, не связанных с IT. Организации часто адаптируют принципы Agile под свои нужды.

В рамках Agile проект делится на небольшие забеги – «спринты», продолжительность которых обычно составляет 2–4 недели

Методология Agile Плюсы

- Владелец продукта участвует во всем процессе разработки ПО, дает обратную связь и помогает определить направление развития проекта.
- Agile-методология учитывает, что требования и приоритеты могут меняться в ходе проекта. Это позволяет вносить коррективы в конце каждого спринта.
- Методология способствует сотрудничеству между командами. Разработчики и дизайнеры работают в тесном контакте, общаются между собой и коллективно принимают решения.
- Обеспечивает тестирование и оценку качества на протяжении всего процесса разработки, а не только в конце. Это помогает устранять ошибки на ранних этапах и быстро создавать качественное ПО.
- Документация ценится, но она не так важна, как разработка качественной программы, которая будет представлять ценность для пользователя.

Методология Agile Минусы

- Гибкий характер модели затрудняет оценку сроков и стоимости разработки на этапе планирования.
- Чтобы все прошло гладко, для реализации Agile-проектов нужна команда, у которой есть опыт совместной работы.
- Для некоторых проектов необходима подробная документация — это может быть нужно для соблюдения нормативных требований. Краткой документации бывает недостаточно.
- Переход на Agile может встретить сопротивление со стороны команды разработчиков и руководства, привыкших к традиционным методологиям.

Методология Agile Кому подходит

✅ Agile-разработка подходит для проектов, требующих гибкости и постоянных обновлений — например, стартапов. Она может стать лучшим выбором для тех случаев, когда важна способность к быстрой адаптации из-за меняющихся требований. Она также идеально подходит для проектов, предполагающих инновации и эксперименты. Что касается размера команды, то методология Agile эффективна для малых и средних команд, которые могут тесно сотрудничать и быстро принимать решения.

❌ Однако Agile может не подойти для команд, у которых нет потребности в коллаборации и коммуникации. Методология не подходит для больших проектов, требующих жесткой структуры и большого количества документации. Не подходит этот метод и для проектов с недостаточным участием владельца продукта, когда до него невозможно дозвониться, чтобы получить обратную связь, или у него просто нет на это времени.

Методология Scrum

Scrum — это система управления проектами, основанная на принципах Agile. Она предназначена для команд из десяти и менее человек. Они разделяют работу на небольшие задачи, которые нужно сделать в течение определенного времени. Как и в Agile, эти временные интервалы называются «спринты». Scrum способствует самоорганизации команды: они решают сложные задачи, анализируют победы и поражения и улучшают работу.

Модель разработки ПО Scrum построена таким образом, чтобы помочь командам естественным образом адаптироваться к меняющимся условиям рынка и потребностям пользователей. В то же время короткие циклы позволяют разработчикам быть эффективнее. Scrum обеспечивает структуру, оптимизирует разработку и при этом остается гибким и учитывает желания владельца продукта.

Методология Scrum



Методология Scrum Плюсы

- Легко адаптировать к нестабильным требованиям проекта, подходит для работы в динамичной среде.
- Продукт отвечает потребностям владельца благодаря его участию в проекте.
- Процесс работы становится прозрачным за счет регулярных встреч и видимого прогресса проекта. Это также упрощает общение между участниками.
- Частые тестирование и анализ помогают выявить и устранить ошибки на ранних этапах разработки.

Методология Scrum Минусы

- Методология подходит только для сплоченных опытных команд, потому что она подразумевает постоянное взаимодействие всех членов команды.
- Может не подходить для проектов с фиксированными сроками или бюджетом, так как подход ориентирован на гибкость и адаптивность.
- Может быть сложной для внедрения и потребовать от команд прохождения обучения, особенно при переходе с водопадной модели.
- Менеджеры, использующие Scrum, могут столкнуться с трудностями при работе с очень большими и сложными проектами.

Методология Scrum Кому подходит

✅ Scrum подходит для проектов, где нужны гибкость и совместная работа. Методология хорошо показывает себя, если требования и сроки меняются или если на рынке жесткая конкуренция. Если владельца продукта заинтересован в процессе разработки и активно участвует в нем — например, дает фидбек на каждом из этапов — такой проект получит выгоду от клиентоориентированности Scrum.

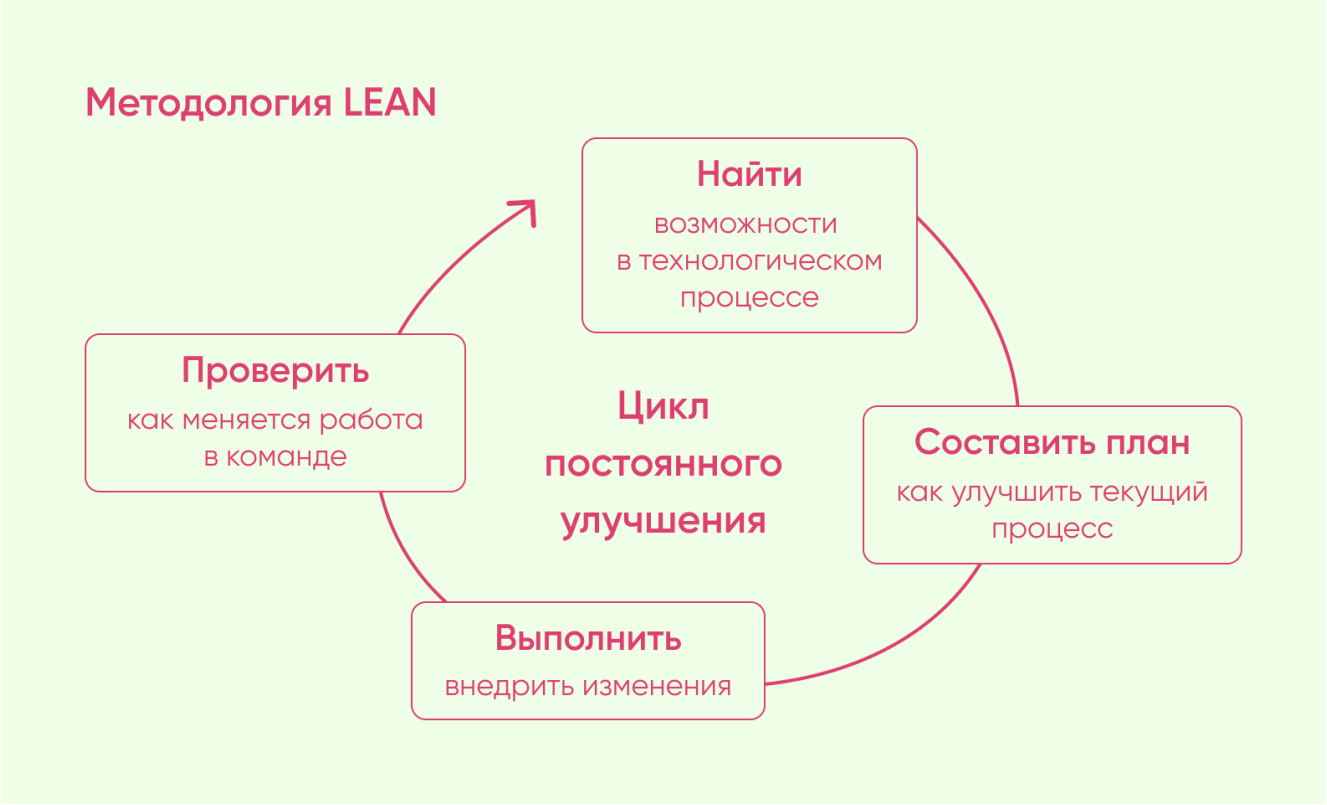
❌ Однако Scrum может не подойти для проектов, требующих строгого соблюдения нормативных требований, и проектов, в которых невозможно поставить даже короткие недельные цели на спринт. Он также не подойдет для проектов без четкой идеи и налаженного пайплайна, а еще если в команде не хватает ключевых скиллов или есть конфликты и другие проблемы.

Методология Lean

Бережливая разработка — еще одна методология разработки на базе Agile. Она направлена на повышение эффективности за счет того, что все лишнее убирается из процесса. Разработчики делают акцент только на том, что необходимо, избегают ненужную работу и тем самым повышают качество продукта.

Если убрать задачи и действия, не приносящие реальной пользы, члены команды достигают оптимальной эффективности. В данном случае к «ненужному» можно отнести дополнительные функции, избыточный код, неэффективные процессы и излишнюю бюрократию. Проект делится на небольшие задачи, которые можно закончить быстро. Это сокращает время работы и упрощает обратную связь.

Методология Lean



Методология Lean Плюсы

- Бережливая разработка направлена на устранение лишних деталей и оптимизацию процессов. Это помогает повысить эффективность и снизить затраты ресурсов.
- Гарантирует, что результат работы соответствует потребностям и предпочтениям владельца продукта. Активный фидбек первых пользователей также повышает качество ПО.
- Бережливая разработка адаптируется к меняющимся требованиям пользователей и условиям рынка.
- Выполнение небольших задач и сдача проекта точно в срок позволяют ускорить запуск и сократить время выхода на рынок.

Методология Lean Минусы

- Как и с другими Agile-методологиями, с Lean-разработкой могут возникнуть проблемы с управлением нагрузкой, если нет строгого требования по объему работы.
- От владельца продукта потребуются регулярные правки и обратная связь, а это может оказаться сложной задачей.
- В очень больших проектах могут возникнуть трудности с полной реализацией принципов Lean-разработки.
- Подход основан на минимальном документировании, это может не подойти для отраслей или проектов, требующих подробной документации.

Методология Lean Кому подходит

✅ Методология бережливой разработки подходит для небольших и средних проектов, где самая важная задача — создать ценный для пользователя продукт и иметь возможность быстро вносить изменения. Она также хорошо подходит для проектов, требующих высокого уровня взаимодействия и постоянного совершенствования. А еще Lean-разработка хороша в тех случаях, когда важно оптимизировать процесс разработки и добиться максимальной эффективности.

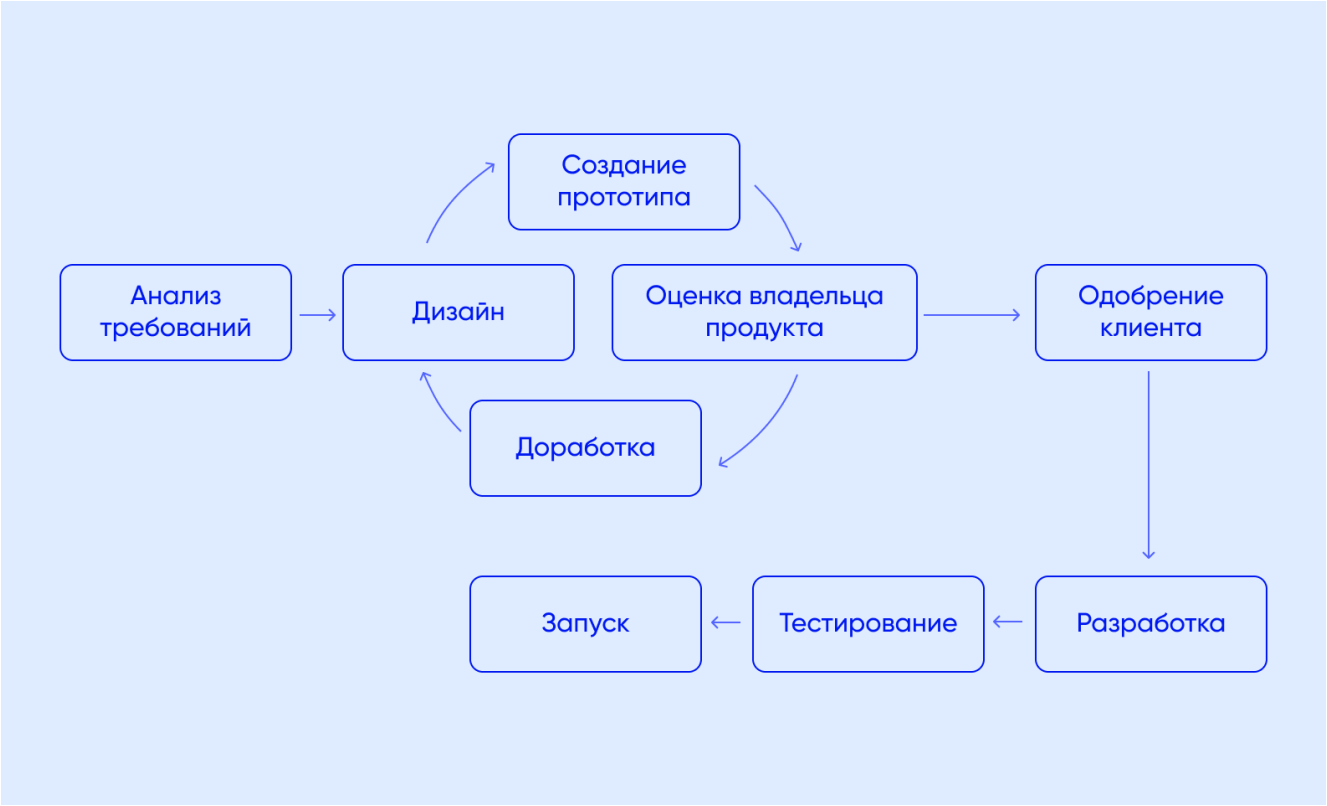
❌ Однако бережливая разработка может не подойти для высокорегулируемых отраслей или больших проектов с жесткими требованиями. Этот метод разработки также может оказаться неудачным выбором для проектов, требующих более структурированного подхода к менеджменту или долгосрочного планирования.

Prototype model

Создание прототипа (Prototype model) — это итеративный подход к разработке ПО. Он предполагает, что команда создаст рабочую модель (прототип) перед тем, как начнет разработку конечного продукта. Эту модель разработки ПО применяют для тестирования и проверки бизнес-идей.

Прототипирование позволяет владельцу продукта на раннем этапе увидеть интерфейс и функции программы. Визуализация помогает прояснить идеи, которые в противном случае останутся абстрактными. Хотя создание прототипа и добавляет еще один этап к проекту, в долгосрочной перспективе это может ускорить разработку. По четким требованиям и дизайну команда работает эффективнее.

Prototype model



Prototype model Плюсы

- Прототип позволяет показать владельцу продукта дизайн и функции приложения на ранних стадиях разработки. Становится проще уточнять задачи и вносить правки.
- Прототипы помогают выявить и решить проблемы до того, как команда инвестирует много сил в разработку. Это снижает риск дорогостоящих изменений на следующих этапах.
- Можно быстро получить обратную связь от пользователей и членов команды. Визуальное представление также снижает риск недопонимания.

Prototype model Минусы

- Создание прототипов может занимать много времени, особенно для сложных приложений. Разработка нескольких версий — тоже не быстрый процесс.
- Прототипы не полностью отражают функции конечного приложения, что может привести к расхождению между ожиданиями и реальностью.
- Разработка и поддержка прототипов может потребовать дополнительных ресурсов, в том числе оплаты труда команды.

Prototype model Кому подходит

✅ Разработка приложения по прототипу подходит для проектов с большим количеством неизвестных, когда команде разработчиков необходимо работать над демо-версией конечного продукта. Это идеальный вариант, когда не требуется подробная документация и основное внимание уделяется обратной связи.

❌ Однако такой подход может не подойти для проектов с фиксированными сроками и ориентацией на соблюдение нормативных требований. Он также может оказаться избыточным для простых проектов с подробно прописанными задачами.

Экстремальное программирование (XP)

Экстремальное программирование — это тоже методология на базе Agile. Для нее характерны командная работа, общение и быстрый фидбек. XP считается одной из самых радикальных форм Agile и сильно отличается от других подходов.

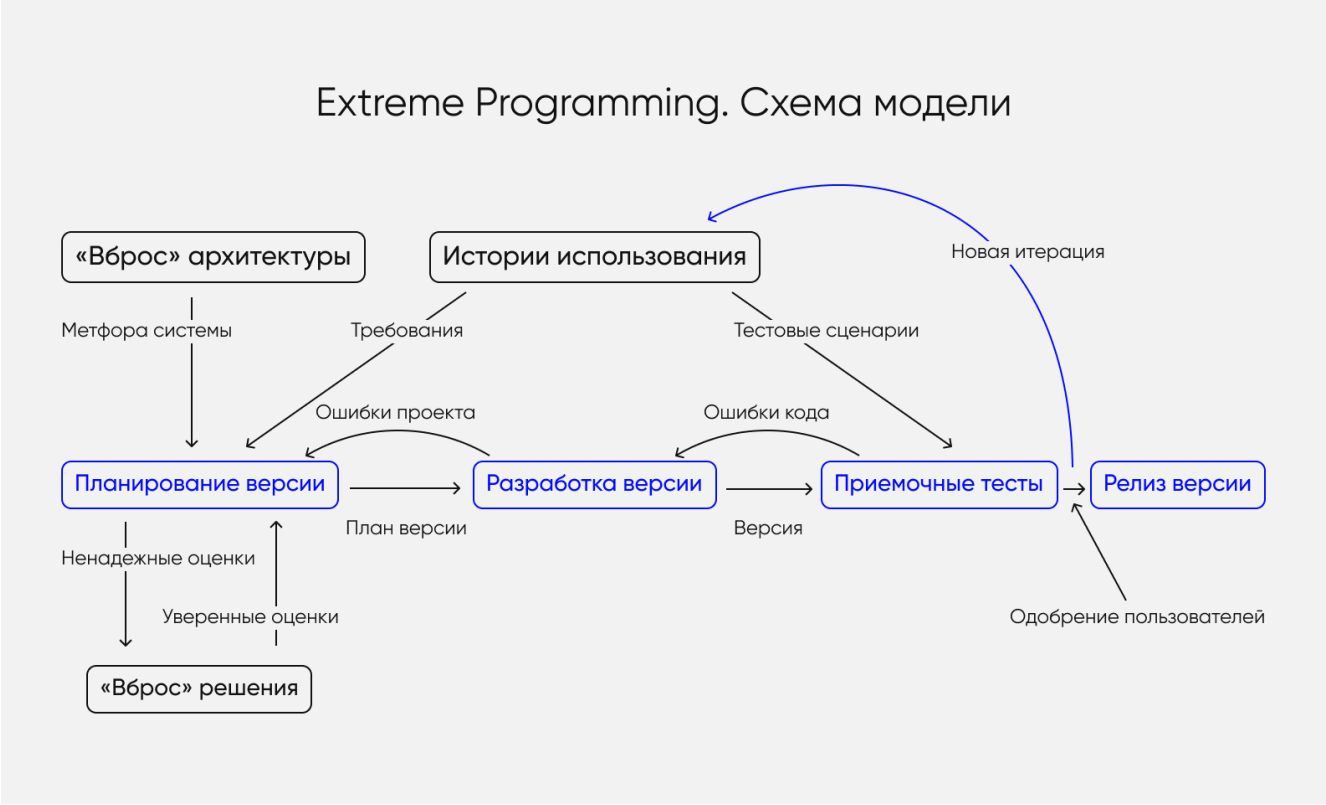
XP делает акцент на клиентоориентированности и побуждает разработчиков ПО творчески подходить к работе. Важную роль в экстремальном программировании играют тестирование и проверка качества кода. Это нужно для того, чтобы избежать ошибок и в кратчайшие сроки запустить качественное ПО.

Экстремальное программирование (XP)

Экстремальное программирование — это тоже методология на базе Agile. Для нее характерны командная работа, общение и быстрый фидбек. XP считается одной из самых радикальных форм Agile и сильно отличается от других подходов.

XP делает акцент на клиентоориентированности и побуждает разработчиков ПО творчески подходить к работе. Важную роль в экстремальном программировании играют тестирование и проверка качества кода. Это нужно для того, чтобы избежать ошибок и в кратчайшие сроки запустить качественное ПО.

Экстремальное программирование (XP)



Экстремальное программирование (XP) Плюсы

- Проекты с использованием XP короткие — обычно они длятся всего несколько месяцев. Это связано с тем, что каждый этап сдается быстро.
- Подход обеспечивает предсказуемый и прозрачный процесс разработки.
- Команда работает сообща для достижения поставленных целей в сжатые сроки. Это позволяет наладить совместную работу и быстро адаптироваться к изменению требований.
- Большое внимание уделяется тестированию, экспертной оценке и проверке качества кода, что снижает риск ошибок.

Экстремальное программирование (XP) Минусы

- Самое главное в экстремальном программировании — этап разработки (создание кода), он может перетянуть на себя все внимание. Из-за этого нередко страдает дизайн.
- Требует большого упорства, творческого подхода и нестандартного мышления, чтобы привести программу в соответствие с потребностями пользователей.
- Предполагается, что в начале проекта у владельца продукта нет четкого представления о результате.

Экстремальное программирование (XP) Кому подходит

✅ XP подходит для небольших и средних проектов — например, когда нужно регулярно получать от конечных пользователей обратную связь и поддерживать высокий уровень взаимодействия между членами команды. Он лучше всего подходит для проектов, ориентированных на создание программ высокого качества. А еще XP может стать хорошим выбором для тех, кто хочет сократить административные расходы.

❌ Однако XP может оказаться не самым подходящим вариантом для проектов в высокорегулируемых отраслях или проектов с жесткими, не подлежащими обсуждению требованиями. Он может не подойти для проектов с фиксированными сроками, где важны документирование каждого этапа и тщательное планирование. Команды, привыкшие к водопадной модели, могут не принять XP, например, из-за парного программирования и частых встреч с владельцем продукта.

Быстрая разработка приложений (RAD)

Быстрая разработка приложений — это итеративная методология, при использовании которой важно разработать продукт быстро и, если необходимо, создать несколько прототипов. Метод Rapid Application Development (RAD) основан на обратной связи от пользователей и совместной работе всех членов команды, что позволяет ускорить выполнение проекта и избежать проблем после запуска.

Работая по модели RAD, команда использует инструменты и фреймворки быстрой разработки и обычно опирается на визуальные среды разработки — они помогают создавать ПО в кратчайшие сроки. В рамках этой модели разработки программного обеспечения, продукт регулярно тестируют. И взаимодействие с пользователями помогает сделать так, чтобы ожидание и реальность совпали.

Быстрая разработка приложений (RAD)



Быстрая разработка приложений (RAD) Плюсы

- Позволяет ускорить циклы разработки, а значит, приблизить время выхода ПО на рынок.
- Программа соответствует потребностям владельца продукта, потому что команда регулярно с ним общается и показывает промежуточные результаты. Это снижает риск последующих дорогостоящих изменений.
- Хорошо адаптируется к изменениям и позволяет учитывать новые пожелания владельца продукта по ходу проекта.
- Легче выявлять и устранять проблемы на ранних стадиях, сокращается время и затраты на разработку.

Быстрая разработка приложений (RAD) Минусы

- RAD может не подойти для больших и сложных проектов, где важно строго соблюдать нормативные требования.
- Без строгого контроля итерации могут привести к тому, что проект превысит запланированный масштаб.
- Эффективность модели сильно зависит от наличия инструментов и фреймворков быстрой разработки, а также опытных разработчиков.
- В условиях ограниченных ресурсов данная методология разработки может оказаться неэффективной и нецелесообразной.

Быстрая разработка приложений (RAD) Кому подходит

✅ RAD удобен для разработки небольших и средних проектов в сжатые сроки. Он хорошо подходит для проектов, требующих быстрого создания прототипов и проверки идей. RAD подойдет для проектов с нечеткими требованиями, требующими обратной связи от пользователей и последующей адаптации.

❌ Однако RAD может не подойти для проектов с ограниченными ресурсами. Когда члены команды параллельно заняты другими проектами, им может не хватить времени работать по RAD. Очень большие и сложные проекты могут не выдержать быстрых итераций — для них нужен более структурированный подход. Проекты в высокорегулируемых отраслях также могут столкнуться с трудностями при внедрении RAD.

Функционально-ориентированная разработка (FDD)

Функционально-ориентированная разработка (Feature Driven Development, FDD) — это гибкая методология, также основанная на принципах Agile. Она направлена на создание небольших функций или функциональных блоков. FDD — итеративная и инкрементальная (пошаговая) методология, и ее цель — быстро получить ощутимые результаты.

Есть пять основных этапов разработки в рамках FDD: создание общей модели, формирование списка функций, планирование по функциям, дизайн по функциям и разработка по функциям. FDD предусматривает отчетность на всех этапах для отслеживания прогресса и фиксирования результатов. Обычно методология используется в крупномасштабных проектах.

Функционально-ориентированная разработка (FDD)



FDD Плюсы

- Используя FDD, процесс разработки можно легко подстроить под требования владельца продукта. Методология позволяет быстро реагировать при изменении приоритетов.
- Функционально-ориентированная разработка делает упор на быстрое создание основных функций.
- Включает эффективные методы управления проектами, например, использование ограниченных по времени циклов разработки. Это позволяет обеспечить достижение поставленных целей.

FDD Минусы

- Не очень хорошо подходит для небольших проектов и для проектов, в которых работает только один разработчик.
- В меньшей степени, чем другие Agile-подходы, ориентирована на непосредственное участие владельца продукта.
- Большая ответственность ложится на плечи главного программиста, который должен одновременно выполнять функции координатора, ведущего дизайнера и наставника для новых членов команды.

FDD Кому подходит

✅ FDD подходит для команд, которые ищут простой, масштабируемый, но структурированный Agile-метод, дающий предсказуемые результаты. FDD удобен для владельца продукта и поощряет ведение подробной документации. Он лучше всего подходит для больших проектов, в которых все же требуется гибкость.

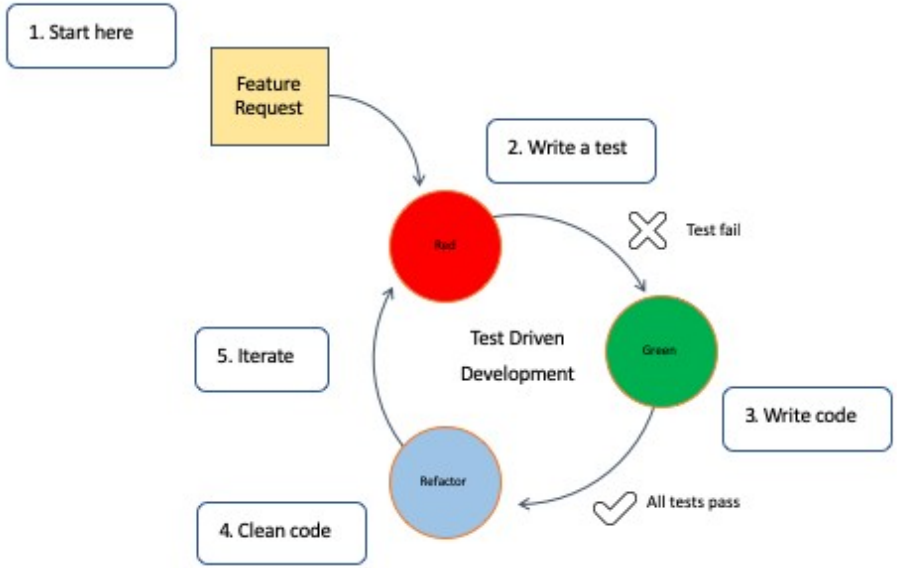
❌ Однако этот метод может не подойти для проектов, требующих более линейного подхода. FDD может внести излишнюю сложность в небольшие проекты с простыми требованиями. Проекты, ориентированные на исследования и изучение новых технологий, тоже не выиграют от применения функционально-ориентированной системы.

Тестирование через проектирование (TDD)

Тестирование через проектирование (Test-Driven Development, TDD) — это гибкая методология разработки программного обеспечения, также основанная на принципах Agile. Ее главная идея — сначала создавать автоматические тесты, описывающие желаемое поведение программы, а уже затем писать код, который сможет эти тесты пройти. Такой подход обеспечивает контроль качества с самых ранних этапов и помогает поддерживать высокую стабильность кода.

В основе TDD лежит короткий цикл разработки: сначала пишется тест, затем — минимальный код для его прохождения, после чего код реорганизуется (рефакторинг). Этот цикл повторяется снова и снова.

Тестирование через проектирование (TDD)



TDD Плюсы

- Повышает качество кода: тесты пишутся заранее, ошибки выявляются до внедрения в релиз.
- Ускоряет выявление дефектов и снижает стоимость их исправления.
- Формирует своеобразную документацию: тесты отражают требования к системе.
- Упрощает рефакторинг и дальнейшее развитие кода, так как всегда есть набор проверочных тестов.
- Развивает дисциплину у разработчиков и способствует системному мышлению.

TDD Минусы

- Требует времени: сначала пишутся тесты, а затем код, поэтому на старте скорость разработки может снижаться.
- Не подходит для проектов с высокой неопределенностью требований. Если цели постоянно меняются, тесты придется переписывать.
- Нужны навыки написания тестов, а у начинающих программистов это вызывает трудности.
- Тесты не всегда покрывают весь код, поэтому часть ошибок может остаться незамеченной.

TDD Плюсы

✅ TDD прекрасно подходит командам, которые работают по четко определенным требованиям и для которых надежность кода критична. Особенно эффективен он при разработке систем, где ошибки недопустимы (например, банковские или медицинские приложения).

❌ Метод неподходит для исследовательских проектов, стартапов на ранней стадии или систем, где требования часто меняются. В таких условиях написание тестов вперед замедляет процесс и не приносит пользы.